

```
# ASSIGNMENT 1
```

```
## Introduction to Web Development
```

```
**Subject:** Web Development\  
**Assignment:** Assignment 1\  
**Topic:** Introduction to Web Development
```

```
## Introduction
```

Web development is the process of creating, building, testing, and maintaining websites and web applications that run on the Internet or an intranet. It involves several technologies and areas such as **HTML**, **CSS**, **JavaScript**, web servers, databases, APIs, and client-server architecture.

Web development is generally divided into three major areas:

- **Frontend Development:** Development of the part of a website that users see and interact with.
- **Backend Development:** Development of server-side logic, APIs, authentication, and application processing.
- **Full-Stack Development:** Development involving both frontend and backend technologies, along with databases and deployment.

Modern web applications are used in almost every field, including education, banking, e-commerce, healthcare, entertainment, and social media.

```
# 1. Difference Between Frontend, Backend, and Full-Stack Development
```

Type	Meaning	Main Technologies	Example
Frontend product Development interface	Creates the user interface and client-side functionality	HTML, CSS, JavaScript, React	Amazon page
Backend an Development	Handles server-side logic, APIs, authentication, and data processing	Node.js, Python, Java, PHP, SQL	Processing online order
Full-Stack	Works on both frontend	HTML, CSS, JS,	Developing a

Development	and backend and often databases/deployment	React, Node.js, Python, SQL	complete e-commerce website
-------------	---	--------------------------------	-----------------------------------

Frontend Development

Frontend is the **client-side** part of a web application. It determines how the website looks and behaves in the browser.

Examples: HTML, CSS, JavaScript, React, Angular, and Vue.js.

****Real-world example:**** The product page, navigation menu, buttons, search box, and shopping cart interface of an e-commerce website.

Backend Development

Backend is the ****server-side**** part of a web application. It processes requests, applies business logic, communicates with databases, and sends responses to the client.

Examples: Node.js, Java/Spring Boot, Python/Django, PHP, and .NET.

```
**Real-world example:** When a user places an order, the backend  
validates the order, checks product availability, stores the order in a  
database, and returns the result.
```

Full-Stack Development

A full-stack developer works with both frontend and backend components. They may also work with databases, APIs, authentication, testing, and deployment.

****Real-world example:**** A developer creates the shopping interface, develops the order API, connects the application to a database, and deploys the complete application.

2. Client-Server Model Diagram

The client-server model allows a client such as a web browser to communicate with a server.

```

''' text
+-----+
| CLIENT |
| Web Browser |
| Chrome/Firefox |
+-----+
                                     HTTP/HTTPS Request
                                     ----->
                                     |
                                     v
                                     +-----+
                                     | SERVER |
                                     | Web Server |
                                     | Application |

```

```

+-----+
|                                     |
|                                     |
|                                     |
|                                     |
|                                     |
|                                     |
|                                     |
|                                     |
+-----+
      CLIENT
Displays Page

\ \ \

HTML/CSS/JS

SERVER
+-----+
Response
v

```

Step 5: Server Processing

The server receives the request and may: - Locate a static file. - Execute application logic. - Query a database. - Call another API.

Step 6: HTTP Response

The server sends a response containing resources such as HTML, CSS, JavaScript, images, or JSON.

Step 7: Browser Rendering

The browser parses HTML to create the **DOM (Document Object Model)**, processes CSS to create styling information, executes JavaScript, and renders the final page on the screen.

```
``` text
URL
|
v
DNS Resolution
|
v
Server Connection
|
v
HTTP/HTTPS Request
|
v
Web Server/Application
|
v
Database/API (if required)
|
v
HTTP Response
|
v
Browser
|
v
HTML + CSS + JavaScript
|
v
Rendered Web Page
```
```

4. Tools Required to Set Up a Web Development Environment

The following tools are commonly required:

| Tool | Purpose |
|-----------------|--|
| VS Code | Writing and editing source code |
| Web Browser | Running and testing websites |
| HTML | Creating webpage structure |
| CSS | Designing and styling webpages |
| JavaScript | Adding behavior and interactivity |
| Node.js | Running JavaScript outside the browser and using npm |
| npm | Managing JavaScript packages |
| Git | Version control |
| GitHub | Hosting and collaborating on code |
| Developer Tools | Debugging and inspecting webpages |
| Local Server | Testing websites locally |

Explanation

****1. VS Code:**** A popular source-code editor used to write HTML, CSS, JavaScript, and other programming languages.

****2. Web Browser:**** Chrome, Firefox, Edge, or Safari can be used to run and test web pages.

****3. HTML:**** Defines the structure and content of a webpage.

****4. CSS:**** Controls colors, layouts, fonts, spacing, and responsive design.

****5. JavaScript:**** Adds dynamic behavior and interactivity.

****6. Node.js:**** Provides a JavaScript runtime useful for modern web development and backend applications.

****7. Git:**** Tracks changes in source code and helps manage different versions.

****8. GitHub:**** Provides remote repositories for storing and collaborating on projects.

****9. Browser Developer Tools:**** Help inspect HTML/CSS, debug JavaScript, analyze network requests, and check performance.

****10. Local Server:**** Allows developers to test applications in a local development environment before deployment.

5. What Is a Web Server?

A **web server** is software, and sometimes the computer running that software, that receives HTTP/HTTPS requests from clients and sends appropriate responses.

For example, when a browser requests a webpage, the web server may

return an HTML file.

Commonly Used Web Servers

1. ****Apache HTTP Server**** -- A widely used open-source web server.
2. ****Nginx**** -- Commonly used as a web server, reverse proxy, and load balancer.
3. ****Microsoft IIS**** -- Microsoft's web server for Windows environments.
4. ****LiteSpeed**** -- A high-performance web server often used for hosting.
5. ****Caddy**** -- A modern web server known for simple configuration and automatic HTTPS.

Basic Working

```
``` text
Browser
 |
 | HTTP/HTTPS Request
 v
Web Server
 |
 | HTML/CSS/JS or application response
 v
Browser
```
```

6. Roles of Frontend Developer, Backend Developer, and Database Administrator

Frontend Developer

A frontend developer is responsible for the user-facing part of a web application.

Responsibilities

- Build webpage interfaces.
- Write HTML, CSS, and JavaScript.
- Create responsive layouts.
- Implement user interactions.
- Connect frontend applications to APIs.
- Test browser compatibility and usability.

Backend Developer

A backend developer handles server-side processing and application logic.

Responsibilities

- Build APIs.
- Implement business logic.
- Handle authentication and authorization.
- Process user requests.
- Communicate with databases.
- Implement validation and security controls.
- Manage server-side errors.

Database Administrator (DBA)

A database administrator manages the database infrastructure and data.

Responsibilities

- Create and manage databases.
- Manage users and permissions.
- Monitor database performance.
- Perform backups and recovery.
- Maintain data integrity.
- Apply security and access controls.

Overall Flow

```
``` text
```

```
User
```

```
|
```

```
v
```

```
Frontend
```

```
|
```

```
v
```

```
Backend / API
```

```
|
```

```
v
```

```
Database
```

```
^
```

```
|
```

```
DB Administrator manages database
```

```
```
```

7. Install VS Code and Configure It for HTML, CSS, and JavaScript

Installation Steps

1. Download and install **Visual Studio Code**.
2. Open VS Code.
3. Create a folder for the web project.
4. Open the folder in VS Code.
5. Create the following files:

```
``` text
```

```
web-project/
```

```
|— index.html
```

```
└─ style.css
└─ script.js
...
```

6. Add HTML code to `index.html`.
7. Add CSS code to `style.css`.
8. Add JavaScript code to `script.js`.
9. Open `index.html` in a browser or use a local development server.
10. Use browser Developer Tools to test and debug the webpage.

### ### Recommended VS Code Extensions

- Live Server
- Prettier - Code formatter
- ESLint (for JavaScript projects)

### ### Sample Setup

```
index.html
```

```
``` html
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello Web Development</h1>
  <button onclick="showMessage()">Click Me</button>

  <script src="script.js"></script>
</body>
</html>
```
```

```
style.css
```

```
``` css
body {
  font-family: Arial, sans-serif;
  text-align: center;
}
```
```

```
script.js
```

```
``` javascript
function showMessage() {
  alert("Hello from JavaScript!");
}
```
```

### ### Screenshot

**\*\*Insert a screenshot of your installed VS Code setup here.\*\***

Suggested screenshot: VS Code open with `index.html`, `style.css`, and `script.js` visible in the Explorer panel.

## # 8. Difference Between Static and Dynamic Websites

| Feature           | Static Website                  | Dynamic Website                  |
|-------------------|---------------------------------|----------------------------------|
| Content           | Usually fixed                   | Can change based on user/data    |
| Server Processing | Little or none                  | Usually required                 |
| Database          | Usually not required            | Often used                       |
| Speed             | Generally fast                  | Depends on processing            |
| Maintenance       | Simple for small sites          | More complex                     |
| Example           | Portfolio or documentation site | Amazon, Facebook, online banking |

### ## Static Website

A static website delivers mostly pre-created files to users. The content is generally the same for every visitor unless the files are changed.

**\*\*Example:\*\*** A simple personal portfolio website containing HTML and CSS pages.

### ## Dynamic Website

A dynamic website can generate or update content based on user input, sessions, database records, or application logic.

**\*\*Example:\*\*** An e-commerce website can show different products, prices, orders, and recommendations based on data.

## # 9. Five Web Browsers and Their Rendering Engines

A **\*\*browser rendering engine\*\*** converts web technologies such as HTML and CSS into the visual page displayed to the user. JavaScript execution is generally handled by a separate JavaScript engine.

| Browser | Rendering Engine | JavaScript Engine |
|---------|------------------|-------------------|
|---------|------------------|-------------------|

|                 |        |                |
|-----------------|--------|----------------|
| Google Chrome   | Blink  | V8             |
| Microsoft Edge  | Blink  | V8             |
| Mozilla Firefox | Gecko  | SpiderMonkey   |
| Apple Safari    | WebKit | JavaScriptCore |
| Opera           | Blink  | V8             |

### ### How Rendering Engines Differ

#### ### Blink

Blink is used by Chromium-based browsers such as Chrome, Edge, and Opera. It is designed for fast and standards-compliant rendering and is closely integrated with the Chromium project.

#### ### Gecko

Gecko is Mozilla's browser engine and is used by Firefox. It has its own implementation of web platform features and standards.

#### ### WebKit

WebKit is used by Safari. On Apple platforms, browser engines are subject to Apple's platform requirements.

### ### Why Engines Matter

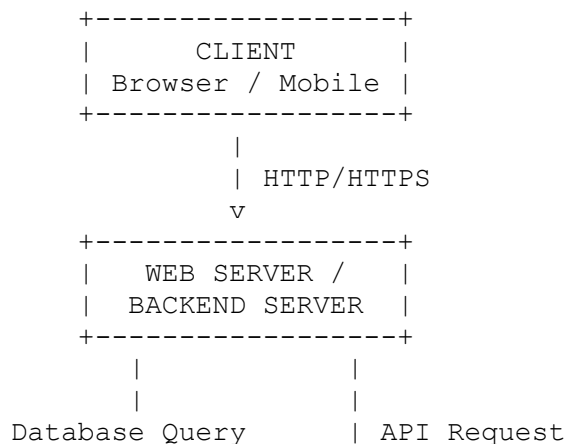
Different rendering engines can have differences in: - CSS feature support. - HTML behavior. - Web APIs. - Performance. - Layout and painting implementation. - Experimental or newly introduced web features.

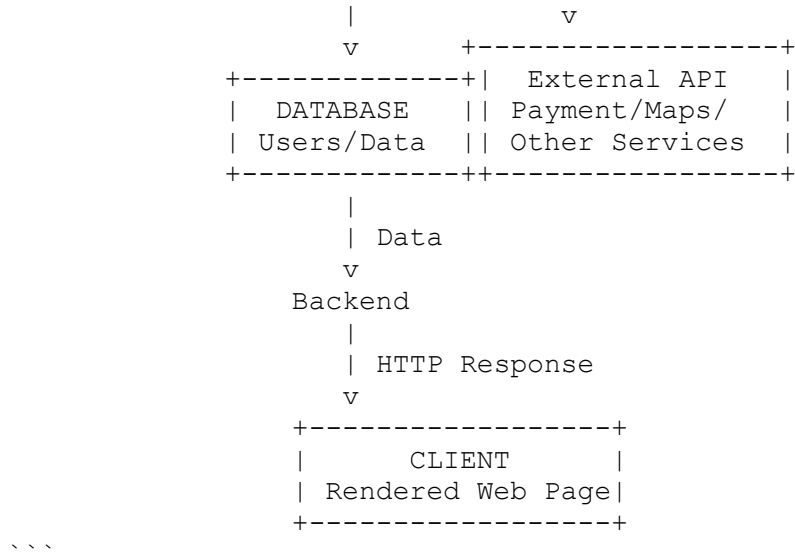
Therefore, web developers should test websites across multiple browsers.

## # 10. Basic Web Architecture Flow: Client, Server, Database, and APIs

The following diagram shows the basic flow of a modern web application:

```
``` text
```





Explanation of Components

Client

The client is normally a web browser or mobile application. It sends requests and displays responses.

Server

The server contains application logic. It handles requests, authentication, validation, and communication with databases and APIs.

Database

The database stores persistent information such as: - User accounts - Products - Orders - Messages - Application settings

APIs

An API (Application Programming Interface) allows different software systems to communicate.

For example, an online shopping application may use: - Payment API - Maps API - Email API - Authentication API

Complete Flow

``` text

Client

|

| 1. Request

v

Server

|

| 2. Query / Process

```

 v
Database
|
| 3. Data
 v
Server
|
| 4. API call (if needed)
 v
External API
|
| 5. API response
 v
Server
|
| 6. HTTP response
 v
Client
```
```

Conclusion

Web development combines frontend, backend, databases, servers, APIs, and development tools to create modern web applications. The browser acts as the client, while web servers and backend applications process requests and communicate with databases and external APIs. HTML, CSS, and JavaScript form the foundation of frontend development, while backend technologies provide server-side functionality.

Understanding the **client-server model**, web servers, browser rendering, databases, APIs, and development environments provides a strong foundation for learning advanced web development technologies.

References

1. MDN Web Docs -- Web development and web platform documentation.
2. W3C -- Web standards and specifications.
3. Visual Studio Code Documentation.
4. Apache HTTP Server Documentation.
5. Nginx Documentation.
6. Mozilla Developer Documentation.